

AI 反诈劝阻 · 项目详细介绍与答辩问答手册

本文用途：给需要完整、可讲、能扛住追问的项目介绍的场景使用——路演前自查、向新同事/评委/合作方讲解、准备 Q&A。内容整合自 PROJECT-INTRODUCTION.md、README.md、CONTEXT.md、CONTEST-POSITIONING.md、SUBMISSION-CHECKLIST.md 以及 tools/ 下四个跑批脚本的源码，新增并重点展开了"实验与验证难度"一章——这是最容易被问穿、也是原有文档里最分散的部分。

全文遵守同一条底线：没验证过的事情，写"没验证过"，不写得含糊。一处越线的主张，会把整份材料的可信度一起打折。

目录

1. 项目定位
2. 要解决的业务问题
3. 核心剧情与人物
4. 产品体验流程
5. 判分机制详解
6. 技术架构
7. 实验与验证难度（重点）
8. 两个类比：帮助非技术听众理解复杂度
9. 常见追问 Q&A（20 问）
10. 当前完成度与路线图
11. 术语速查表
12. 关键文件索引

一、项目定位

用户自己正要清仓或大额转出的那一刻，券商 App 里弹一次三分钟对话：让他去劝一个和他处境一模一样的人。

这不是投顾话术训练器，不是反诈科普长图，也不是一个披着 AI 皮的选择题游戏。它是装在券商 App 里的一次行为干预：触发条件是用户自己账户上的一个异常动作（清仓、大额转出、与历史行为显著背离的赎回），干预方式是角色对调——不告诉用户"你可能被骗了"，而是让他去扮演劝阻者，去劝一个正在经历同一种骗局的虚构角色（老陈）。

1.1 为什么是角色对调，不是更直白的风险提示

传统风险提示弹窗有一个从设计上就无法绕开的缺陷：它要求用户先承认自己可能正在被骗，然后才谈得上往下看提示内容。而恰恰是正在转账的那个人，最不可能在那一刻承认这件事——他刚刚说服了自己"这次不一样""我了解情况""专家说的没问题"，一个弹窗此时跳出来"这可能是诈骗"，效果往往适得其反：用户会把弹窗和劝阻方一起归类为"不理解我、只想拦着我赚钱的人"，防御心理反而加固。

角色对调把这笔认知成本整个挪走了：他不需要承认任何事，他只需要去劝别人。屏幕对面那个人（老陈）和他年龄相仿，进了同一种荐股群，被同一个“内部消息”故事影响，正在经历同一种“再等一天就来不及了”的催促。玩家在劝老陈的过程中，会不自觉地把老陈用的每一个借口、每一次自我说服的逻辑，拿来对照自己此刻的处境。他劝不动老陈的那一刻，会自己想到自己。

这不只是一句漂亮话，背后有一条具体的外部事实支撑：这类荐股/杀猪盘骗局往往会要求受害者签署“保密协议”，明确禁止向证券公司、监管部门、公安机关透露任何信息（来源：证券时报、中证协相关报道，详见 [EVIDENCE.md](#) 第六节）。与此同时，《反电信网络诈骗法》第十八条给银行与支付机构准备的处置动作是“核实交易情况、重新核验身份”——**这两者都是问答式的核验**，而对面那个正在转账的人，早已被骗局一方预先训练成会在核验时撒谎。角色对调不要求受害者披露任何信息、不要求他承认任何事，是这套“预先训练好的谎言防御”目前唯一绕得开的形态。

需要立刻说清楚这条论证的边界：它解释的是“为什么问答式核验会失效”，不能证明“所以角色对调一定会成功”——后者到今天为止是零次测量，这一点会反复出现在下文的“实验与验证难度”一章里，请不要在任何场合把这两句话混着讲。

1.2 责任分工：谁负责什么

产品的核心工程决策写成了 [ADR-0001](#)：AI 只负责把对方“演活”，不决定输赢；给定命中标签之后，信任度增减多少、最终落在哪种结局，完全由程序规则确定。这条分工不是性能优化，是可信度设计——如果分数由模型当场决定，那么“分数会不会被措辞、语气、模型心情影响”这个问题永远说不清；分数一旦交给纯函数的规则表，它就变成了可以离线重跑、可以蒙特卡洛验证、可以在完全不联网的情况下用单元测试锁死的东西。

代价也要说清楚：判分算术可复现，不等于整条链路可复现。“这句话命中了哪把钥匙”这个标签本身仍然由模型判定，而模型分类器不是确定性的——详见第七章。

二、要解决的业务问题

2.1 接哪个信号

产品要响应的不是“用户正在转账”这个动作本身，而是三类带阈值或背离条件的异常：

- 清仓：持仓一次性归零；
- 大额转出：单笔转出金额占账户资产比例超过阈值；
- 与历史行为显著背离的赎回：例如五年只做定投、从无主动交易记录的账户，突然一次性全部赎回。

这三条信号不是这次新写的产品需求——它们已经写在代码里，`app/scenario.py` 的 `Scenario.facts` 中那几行标了 `warn=True` 的事实（比如“今日转出 ¥100,000，占账户资产 78%”“五年零主动交易记录，昨天一次性清仓”），正是照着这三条信号写的剧本细节。旧的产品定位下，这些细节只是营造真实感的布景；这次转向之后，它们被重新赋予了业务含义——变成了真实产品里应该拿来触发判断的条件。

之所以特别强调“异常”这个限定词，是因为最容易被问穿的一句反问就是“用户想清仓很正常，难道要拦住每一个人吗”——答案是不会，触发条件从来都带着阈值和行为背离这两层过滤，不是“只要用户想转账就弹一次对话”。

2.2 归哪条业务线

这件事归属于用户风险与合规条线，不是培训条线，也不是法定合规义务。

真正的推力是投诉与声誉压力,而不是法律条文:某券商 2025 年接到的假冒公司名义作案的线索超过 1900 条,同比增长 280%,涉案金额约 3000 万元,总部当年接待受骗投资者上门 150 人次(来源:证券时报/中证协报道,详见 EVIDENCE.md 第六节)。这组数字本身不需要任何法律解释就能成立,是评委席、合规负责人、业务负责人都会认的真实痛点。

法规层面需要非常精确地划清一条线:《反电信网络诈骗法》全文逐条核对下来,证券公司一次都没有被点名。第十八条("核实交易情况、重新核验身份")与第三十四条(公安机关会同金融/电信/网信部门组织建立预警劝阻系统)的义务人是银行业金融机构与非银行支付机构,不包括证券公司。第四十条的罚则同样只罚银行与非银行支付机构。材料里可以写"呼应""对齐"这类词,绝不能写"依据某法本公司必须做"——被问到"券商有法定义务吗",标准答案就是"没有,这是主动做的"。

2.3 怎么衡量

核心指标是24 小时内转出放弃率 / 撤单率,干预组对照对照组(现行的风险提示弹窗,不是"什么都不给")。护栏指标包括:完成率、弹出后直接关闭率、30 天内二次触发率。

有一个措辞上的精确校正必须记住:指标的口径是"触发干预后 24 小时内,该笔转出没有再次被完成",而不是"取消率"——银证转账是实时划转,一旦执行就没有"撤销"这个动作可言,把指标叫成"取消率"在业务上是不成立的表述(详见 EVIDENCE.md 第二节)。

为什么一定要有对照组:触发异常交易信号的用户里,本来就会有一部分人自己反悔、自己放弃转账。没有对照组,干预组的放弃率就是一个无法解释的数字——它既证明不了干预真的有效,也没法阻止有人拿着这个数字去错误地宣称"有效"。这条设计要求写在 POSITIONING.md 里,是这个项目在效果验证这件事上唯一站得住脚的立场。

三、核心剧情与人物

3.1 老陈的完整故事

老陈,52 岁,在一家本地工厂干了 20 年。三个月前,他被拉进"启航财经·内部交流群",跟着自称有"内部消息"的"王老师"做短线操作。账面上一度出现过盈利,这让他逐渐相信对方真的掌握着别人没有的信息渠道。

出事当天,王老师要求老陈在下午三点前,把 30 万元转到指定账户"统一操作"。这 30 万元本来是准备给孩子办婚礼用的定金,一直放在老陈自己的银行卡上——这一点很关键,因为投资顾问看不见这笔钱,他能看到的只是证券账户里的资金动向。老陈今天早上已经从证券账户先划走了 10 万元,账户上只剩两万八,而且这一切都瞒着家人,也瞒着自己的投资顾问。

玩家扮演的李经理是老陈的开户投资顾问,他收到的只是一条资产异动提醒——这是营销条线常规的"资产流失预警",不是风控报警,因为银证转出到本人绑定卡是账户里最正常不过的操作。李经理起初完全不知道启航财经、王老师、30 万元、下午三点这些细节,必须靠 12 轮对话,一点一点从老陈嘴里把真相挖出来。这个信息差不是叙事上的偷懒,它就是玩法的核心:挖得越准、问得越具体,越能触发"扎根"加成(详见第五章)。

3.2 首页设计的一次重要调整

2026-08-17 之前,产品首页是老陈的微信会话列表——六条会话,把整局要挖的信息一次性摊给了玩家看。这直接导致了一个可预见的产品缺陷:玩家在第 1 轮就能说出"你女儿已经查过那家公司了",而老陈其实还什么都没说。这是一台"未扎根打法生成器",玩家靠提前看到的剧透就能拿高分,判分设计的核心机制(扎根)因此形同虚设。

现在的首页是李经理的工作台(企业微信 + 券商 CRM 风格):一条资产异动预警 + 一张客户档案,每一行信息都是投顾在自己系统里真实能看到的東西。老陈的秘密——王老师、启航财经、30 万元——一个字都不在首页出现,那是要在这一局里问出

来的东西。

原本首页上的六条会话,现在挪到了**复盘**环节,对局结束后逐条揭晓,判定标准是"老陈自己在这一局里说没说过"这句话:

会话来源	透露的信息
启航财经·内部交流群	王老师用"三点前统一操作"制造时间压力
银行短信	老陈今天已经划走 10 万元(这条开局时投顾就已经知道,不计入揭晓分母)
老伴	这笔钱与孩子的婚礼定金有关,家人完全不知情
女儿小雨	已经查过这家公司,直接劝阻过老陈,但没有成功
国家反诈中心	官方预警短信早就发到老陈手机上,被他直接划掉了

最后两条是全局最重要的教学点:"这是诈骗"这四个字,老陈的女儿说过,国家反诈中心也已经通过官方短信说过。空口断言之所以在判分里要扣分,道理就在这里——不是提前警告没有用,而是玩家打完一局之后才发现,自己想说的那句话别人早就说过了,而它没有起作用。这一点也直接回应了"为什么不干脆做个反诈科普弹窗"这个常见质疑。

3.3 核心人物一览

人物	身份与目标	在项目中的作用
李经理	玩家扮演的角色,老陈的开户投顾	发现账户异动后发起劝阻;受"只能问、不能荐"的执业约束
老陈	AI 扮演的被劝阻对象	已被骗局影响,但不是"敌人";会回避、反问、发火、动摇,也会引用自己的真实经历为自己辩护
王老师	骗局中的荐股"老师"	骗局权威来源,每 3 轮在群里再次催促,构成持续的反向施压
老伴	老陈的配偶	完全不知道老陈正在转钱,代表这笔钱原本承担的家庭责任
小雨	老陈的女儿	已经查过启航财经并劝阻失败,说明"简单揭露骗局"本身不足以改变对方的决定
风控系统	触发资产异动提醒的业务系统	给李经理一个合理、合规的介入起点,也限定了他一开始能掌握的信息量
启航财经群	骗局组织与群体环境	提供从众压力、虚假盈利反馈、限时催促,是与李经理争夺信任的另一端

3.4 24 个人格变体、5 个场景

全产品共有 **24 个人格变体**,分属 **5 个场景**:荐股群(老陈,8 个变体)、冒充公检法(周淑琴,4 个)、杀猪盘(老刘,4 个)、刷单返利(月娥姐,4 个)、虚拟币量化(顾之然,4 个)。5 个场景对应骗局利用的 5 种不同心理弱点——荐股群靠"贪"、冒充公检法靠"怕"、杀猪盘靠"情"、刷单返利靠"困"(生活困顿)、虚拟币量化靠"负"(负面情绪或报复心理)。

每局根据一个确定性的对局 ID 选定一种人格,同一局内始终保持一致。所有变体共享同一套剧情骨架(30 万元、下午三点、王老师、已转 10 万元、女儿与老伴、家庭用途等),只在个人经历、心理支点、语言习惯上做变化。以老陈这一组的 8 个变体为例:

变体	背景	心理支点	表达特点
钳工	本地机械厂退休钳工,曾成功提现 8 千元	"我真提过现,骗子不会让我提现"	句子短,偶尔带方言词
电工	吉林齿轮厂电工,尚未提现	对无法提现已有疑虑,但不愿承认	爱反问
修理工	潍坊纺织厂修理工,老伴常年吃药	家庭医疗与婚礼资金双重压力	直来直去
车工	常州五金厂车工,提现过两次	自认懂行,讨厌被当外行	语气不重但很硬
焊工	绵阳汽配厂焊工,账面盈利较高	相信王老师的"证券公司背景"	语气有劲
司机	洛阳水泥厂货车司机	时间压力和隐约不安	爱用省略号,话说一半
磨工	宝鸡轴承厂磨工,瞒着老伴	最怕事情被家人知道	防御最强,先质疑对方动机
装配工	本地机床厂装配工	对退群失联的人有所怀疑	语速感较慢,爱重复后反问

这条骨架共享是一条硬约束,不是随意的写法:如果某个变体反复提到女儿这类信息,会系统性地拉高整体的"扎根率",让蒙特卡洛跑批数字失真——所以像小雨、老伴、账户细节这些高频复用的信息,一律写在骨架里,不写进任何单个变体的专属台词。人格差异只影响演绎的语气和细节,不改变判分规则本身,这样既能让每局的体感不同,也不会让某种人格天然比别的更容易得分——这一点在第七章的蒙特卡洛验证里会看到具体的证明方式。

四、产品体验流程

冷开场(李经理工作台:资产异动提醒 + 客户档案,无剧透)

→ 可跳过的一屏:"你手里有这七种问法"(只给动作名称,不给时机与分值)

→ 12 轮自由文本对话

每轮玩家发一句话 → AI 并发跑两个任务:

· 演绎:生成老陈这一轮的回应

· 分类:把玩家的话归入"钥匙/失误"闭集,并判断是否引用了上一轮内容

安全层逐句检查后流式下发台词;规则引擎据分类结果计算信任度变化

→ 达到终局条件(信任度触顶/触底,或满 12 轮)

→ 结局:先由规则引擎判定,再由 AI 结合全局历史生成收尾台词,前端呈现转账凭证/拒收提示

→ 复盘:逐轮时间线、信任度 K 线、命中的钥匙与失误、错过的追问窗口、六条会话逐条揭晓

→ 分享卡出图(时机对照+统计+K线+信任度百分位,整块出图)

4.1 冷开场

玩家首先看到的是工作台界面,而不是教程:一条资产异动预警加一张客户档案。骗局本身一个字都不出现——那是这一局要问出来的东西。系统会在玩家浏览这个界面的同时,后台悄悄发起开局请求,降低进入聊天页之后的等待感。

正式开打之前有一屏可跳过的引导,列出"七种问法"的名称和一句话说明,故意不给出时机与分值信息——因为词汇本身是可以提前学的课程,而"在什么时候用哪一招"才是这一局真正要考的东西。提前把分值摊开,只会让玩家把游戏玩成"背分表",而不是练习判断时机。

4.2 十二轮自由对话

玩家以李经理的身份用自然语言输入,每一次玩家发言加上老陈的一次回应构成一轮,上限 12 轮。对局过程中,界面只呈现老陈当前的情绪档位、剩余轮数和一些剧情事件,不直接展示命中了哪把钥匙、拿了多少分——避免玩家一边打一边照着屏幕上的实时反馈去逆向推导"标准答案"。

4.3 AI 双路处理

每一轮系统并发执行两个独立的模型调用:

- 演绎:根据老陈的人格设定、完整对话历史、当前情绪档位,以及王老师是否在这一轮催促,生成角色的自然语言回应;
- 分类:把玩家这一轮说的话,判定进"七把钥匙/三项失误/两条合规红线"这个闭集里,同时判断这句话有没有引用上一轮老陈说过的具体内容(即是否"扎根")。

演绎的回复以短句形式先出现在界面上,分类耗时被这段输出时间自然掩盖,玩家感知不到分类在后台跑。老陈这一轮的语气,用的是上一轮结束时的情绪档位——这意味着玩家说中了要害,老陈本轮可能依然嘴硬,要到下一轮才表现出松动。这是刻意保留的真实感,不是系统延迟造成的缺陷:现实中人的态度改变从来不是瞬间的。

4.4 结局与复盘

终局判定完全交给规则引擎,再由 AI 结合整局的对话历史生成最后几句收尾台词。前端通过转账凭证、拒收提示和具体金额来呈现结果。随后系统给出:

- 结局档位与保住的金额;
- 信任度变化的 K 线图;
- 逐轮的话术、命中标签、得分和该轮结束时的信任度;
- 该轮的情绪档位与话术效力倍率;
- 是否抓住或错过了追问窗口;
- 本局未使用的钥匙、以及触发过的失误;
- 若 Redis 可用,展示全局的结局分布和方法命中率统计;
- 分享卡:结算卡、三栏统计、K 线图、信任度百分位整块出图。

五、判分机制详解

5.1 信任度与情绪档位

一局开始时信任度为 32,范围 0—100。达到 80 即"劝住";降到 0 则"被拉黑"。

信任度区间	情绪档位	老陈的典型表现
0—24	戒备	质疑动机,进行防御性反击
25—44	烦躁	不耐烦,想尽快结束对话
45—64	动摇	开始追问骗局细节,露出破绽
65—100	松动	犹豫并主动透露原本不愿说的信息

5.2 七把钥匙与两条合规红线

七把被验证有效的劝阻手法,命中即推高信任度。最初上线的是前三把,2026-08-17 补齐后四把:

钥匙	基础分	业务含义	更合适的时机
锚定真实用途	25	让客户自己说出这笔钱原本的真实用途	戒备、烦躁阶段最有效
苏格拉底式提问	18	不直接给结论,靠提问引导客户自己核实与推导	各阶段都可使用
指出内部矛盾	23	把客户自己前后说过的两个冲突事实并置	动摇、松动阶段最有效
反映式倾听	—	复述并确认对方的情绪,而不是直接反驳	见效力矩阵
支持自主	—	明确表示决定权在客户自己手里,不施压	见效力矩阵
确认理解	—	让客户自己复述一遍骗局的操作流程	见效力矩阵
有据告知	—	给出具体理由的风险提示,而不是空口下结论	仅在动摇/松动阶段才不退化

分类器每轮最多认定一把主要钥匙,防止一句复杂的话被反复计分。**"有据告知"是唯一一把带"退化"机制的钥匙**:如果玩家在戒备或烦躁档位就说出"我认为这是诈骗,理由是……"这类话,系统会把它退化判定为"空口断言"——因为同一句道理,说早了就是说教和断言,说晚了(对方已经开始动摇)才是真正有效的告知。这正是判分系统里"时机比内容重要"体现得最直接的一处设计。

另外有两条**合规红线**——无证荐股、承诺收益——命中即触发重罚。它们的存在是因为局内的李经理身份是持牌投资顾问,"只能问、不能荐"是真实的执业约束,判分表里没有一把钥匙是"给建议",正是这条约束的直接体现。

5.3 三项话术失误

失误	基础分	典型问题
责骂	-4	否定人格、讽刺或贬低客户
说教	-2	长篇摆道理和数据,让客户感到被教育
空口断言	-2	只说"这是诈骗",不给理由或验证路径

失误还会抬高"防御姿态",使接下来几轮的有效话术打折,因此一次失误的影响不只发生在当下这一轮。**只有责骂能把信任度推到"被拉黑"那一档**——说教与空口断言带一条分数地板(2026-08-25 加入),扣分照扣,但不会把对话推向断绝沟通,理由是这两者不是人身羞辱,只是"这一轮没能劝动他",与合规红线的地板设计是同一个逻辑。

5.4 让"时机"真正决定结果的机制

这一部分是整套判分系统里最核心、也是外部最难一眼看懂的部分——它们共同回答一个问题:**为什么同一句话,说的時候不一样,分数能差好几倍。**

- **扎根**:一次命中是否引用并使用了老陈上一轮说过的具体人物、数字、时间或说法。未扎根的命中只按 0.45 倍生效——这意味着套用一句通用模板和真正接住对方原话之间,分数能差一倍以上。
- **钥匙钝化**:同一把钥匙被反复使用时,效力按 1.0、0.7、0.45、0.25 逐次下降,逼迫玩家不能只靠一招走天下。
- **效力矩阵**:每把钥匙在每个情绪档位上都有不同的乘数。例如"指出内部矛盾"如果用得太早,只值 0.3 倍;等到对方进入"松动"档位再用,能达到 1.4 倍。

- **信任流失:**每轮默认下降 2 点,表现的是骗局一方仍在持续影响客户,玩家不动作,信任度也会自然流失。
- **反向施压:**每 3 轮,王老师在群里再次催促,该轮额外下降 3 点信任度,并同步改变老陈的语气变得更急。
- **蓄势池:**前 3 轮信任度封顶在 50,超出上限的部分最多暂存 10 点,从第 4 轮起才逐轮释放——防止玩家靠开局碰运气讲对一句话就快速通关。
- **防御姿态:**责骂、其他失误以及过早拆矛盾都会削弱后续钥匙的效果,并逐轮自然恢复。
- **追问窗口:**老陈首次进入"动摇"或"松动"档位时,会开放两轮机会;如果玩家及时抓住继续有效追问,能获得 1.25 倍加成,一旦错过则倒扣 6 点信任度,并重新变硬。
- **最后一公里:**信任度超过 50 之后,越接近 80,同一句话继续推动信任度的幅度就越小——让老陈心软是便宜的,让他真正说出"我不转了"不便宜。

5.5 结局阶梯

结局	触发方式	业务表现
劝住	任意轮信任度达到 80	老陈放弃转账,30 万元一分未转
拦下	第 12 轮结束时处于松动档位	只转出 2 万元试水,保住 28 万元
拖住	第 12 轮结束时处于动摇档位	当天暂不转账,但尚未真正改变决定
转账	第 12 轮结束时仍处于烦躁或戒备档位	30 万元全部转出
被拉黑	任意轮信任度降到 0	沟通关系中断,提前结束,不纳入四档阶梯

有一处排序刻意反直觉:单看金额,"拖住"(一分没转)本该排在"拦下"(已转出一部分)之上,但产品设计上仍然把"拖住"排在更低的位置——因为"我再想想"多半只是打发人的话,不代表客户已经真正改变了行为;真被说动的表现是行为发生了改变,而不是嘴上推迟。

六、技术架构

6.1 分层

层次	实现	责任
前端	原生 HTML、CSS、JavaScript,零构建	微信式交互、POST SSE 解析、对局展示、复盘、K 线和分享卡
Web 服务	FastAPI + Uvicorn	开局、轮次事件流、健康检查、统计接口和静态资源
对局编排	app/engine.py	并发演绎与分类、降级、事件顺序、历史记录和终局生成
模型网关	app/gateway.py	角色提示词、分类提示词、并发闸门和 provider 协议隔离
模型协议	OpenAI Chat Completions / Anthropic Messages	支持内网网关或公网模型,启动时选择,不在运行期自动切换
判分引擎	app/scoring.py(655 行)	纯函数规则求值、情绪档位、追问窗口和结局判定
安全层	app/safety.py	注入吸收、舞台指示清理、真实证券标的/收益承诺/联系方式拦截
状态令牌	app/state_token.py	客户端持有完整对局状态,服务端用 HMAC 防篡改
统计旁路	Redis(可选)	总局数、结局分布、钥匙和失误命中率;故障不影响对局

6.2 一轮数据流(概述)

一轮对局请求发出后,后端并发向"演绎"模型和"分类"模型各发一次调用:演绎侧持续流式返回台词片段,经安全层逐句过滤后以 sentence 事件推给前端;分类侧返回命中标签和是否扎根,交给规则引擎算出这一轮的信任度变化,以 score 事件推给前端。若达到终局条件,再补发 ending 事件,最后统一发出新的签名状态令牌和 done 事件收尾。

6.3 六条关键工程决策

1. **判分权归程序**:模型只做角色演绎和闭集分类,不接触分值、阈值与胜负条件。
2. **台词落后分数一轮**:并发设计降低了延迟,也恰好符合"人的态度不会瞬间改变"这个现实。
3. **客户端持有签名状态**:重新部署服务不会清空进行中的对局;令牌有效期 2 小时。
4. **按句缓冲后再下发**:先完成安全检查,再把完整短句显示到页面上,防止半句危险内容闪现。
5. **provider 不自动故障转移**:是否把用户输入发送给公网模型,必须由人工显式决定,不能自动切换。
6. **降级不让对局断裂**:演绎失败时使用分档兜底台词,分类失败时按中性判分继续,终局生成失败时使用预置收尾——任何一个模型环节挂掉,对局都不会中断,只是台词质量会下降。

6.4 对外接口

方法	路径	作用
GET	/	打开前端应用
POST	/api/game/start	创建一局,返回开场白、初始状态、人格一致的签名令牌
POST	/api/game/turn	提交一轮话术,通过 SSE 返回 meta → sentence* → score → ending? → state → done
GET	/api/stats	返回全局统计;Redis 未配置或不可用时返回 available: false
GET	/healthz	轻量健康检查,不依赖外部模型和 Redis
GET	/healthz?probe=1	在健康检查基础上探测当前模型 provider
GET	/api/demo/stream	不调用模型的 SSE 部署链路验证

6.5 安全、合规与可靠性

- **输入注入吸收**:识别"忽略系统指令、输出提示词、扮演 AI"等操纵意图后,直接用老陈听不懂的方式回应,不向模型转发攻击内容;
- **输出内容约束**:拦截真实股票代码、上市公司名、具体时间窗加收益百分比、手机号、微信号、二维码和外链;
- **角色一致性**:删除括号舞台动作,丢弃角色标签、英文思考过程和不是老陈会说出的内容;
- **金额与证券代码区分**:剧本中的 50000、300000 不会被误当成股票代码,而证券语境中的真实代码仍会被拦截;
- **请求边界**:玩家输入前端限制 120 字,服务端硬限制 200 字;状态令牌上限 32 KB;
- **防篡改与重放**:状态令牌使用 HMAC-SHA256 签名;单进程内记录已完成轮次的令牌指纹,防止顺手回档重试;
- **密钥管理**:模型密钥和状态签名密钥只来自 .env,缺少 STATE_SIGNING_SECRET 时服务拒绝启动。

七、实验与验证难度(重点章节)

这是整份介绍里最容易被简化、也最容易被问穿的部分。这个项目实际上分成**四层**,每一层"能不能被验证"和"验证起来有多贵"完全不同,把它们混为一谈,是回答"你怎么知道这个东西有效"这类问题时最常见的失分点。

7.1 四层验证的全貌

第一层	判分规则表	纯函数,零模型调用 → 蒙特卡洛平衡模拟(秒级)
第二层	模型分类准确率	需要真实调用模型 → 人工标注集跑批(几分钟,花钱)
第三层	模型演绎质量	需要真实调用模型 → AI 味/重复率跑批(十几分钟,花钱)
第四层	真实干预效果	需要接入真实交易系统 → 至今零次测量,无法用这个仓库单独完成

越往下,验证成本越高、周期越长、越难由这个仓库单独完成。这条梯度本身就是这个项目"能拍胸脯说什么、不能拍胸脯说什么"的分界线。

7.2 第一层:判分规则表——蒙特卡洛平衡模拟

判分引擎是纯函数,这意味着"游戏难不难、平不平衡"这件事不再需要靠人反复试玩去猜,而是可以变成一个可以直接求解的参数搜索问题。tools/balance_sim.py 做的就是这件事:构造 5 种不同能力水平的虚拟玩家("人设"),让每种人设按照自己的行为模型打 20000 局,统计结局分布,再拿一组数值门槛去检验这套参数是否合理。

5 种人设分别是:

人设	占比	特征
novice(新手)	25%	命中率低、失误多,不会轮换钥匙,不认识后四把"学来的"钥匙
average(普通)	40%	中等命中率,会轮换钥匙,但不看情绪档位挑时机
expert(高手)	20%	高命中率、高扎根率,会按当前情绪档位挑效力最高的钥匙
speedrun(极限)	3%	几乎不失手的理论上限人设,专门用来测"游戏有没有一个必胜套路"
parrot(复读攻略)	12%	照抄网上攻略机械复读固定句子,命中率高但几乎不扎根,专门用来测"攻略传开之后游戏会不会当场报废"

这五种人设的行为参数(命中率、扎根率、失误率等)不是随便拍的——文档里明确写着"这是重建出来的,不是原始数据",因为原始的用户行为脚本已经丢失,现在的门槛守的是一个合理区间,不是某一个精确的点估计。

验收门槛本身是从约 500 组参数扫描中反推出来的帕累托前沿,不是拍脑袋定的数字:

门槛项	数值	不满足意味着什么
加权总体胜率	12%—22%	太高说明整体太简单,太低说明大多数人打不通
expert 胜率上限	≤60%	超过说明"会读人的玩家"稳赢,游戏没有难度可言
speedrun 胜率上限	≤85%	超过说明完美执行也不该有输的可能,最优解一旦被摸清就没得玩了
parrot 胜率上限	<15%	超过说明攻略一旦在社交平台传开,游戏当天就报废
整体被拉黑率上限	<12%	超过说明太多人会中途出局,影响传播转化率
落在最差档(转账)的比例上限	≤60%	超过说明大多数人打完只拿到一张"发不出去"的分享卡

这套门槛真实地反映过一次设计缺陷:早期版本里,expert 人设的胜率是 98.4%、平均只需要 5.5 轮就能通关——"会玩的人稳赢",游戏完全没有难度,后来才补上了 expert/speedrun 的胜率上限这两条约束。这说明蒙特卡洛验证不是走过场的自证,它真的会拦下不合理的参数配置。

balance_sim.py 还有一个专门用于跨场景验证的功能:计算每个场景在每个情绪档位上"期望收益最高的钥匙",并两两比较不同场景之间这份"最优解表"的差异档数。这是"五个场景共用一张判分表,但不是简单换皮"这句核心主张唯一能被直接验证的地方——tests/test_balance.py 里有专门的测试守着"任意两个场景的最优解至少 2/4 档不同",一旦某两个场景的最优解完全一样,就说明后一个场景没有存在的必要。

这一层的验证成本几乎为零:零网络调用、秒级完成、可以放进 CI 每次 push 自动跑一遍,这是四层里唯一一层可以做到"完全确定性、随时可重跑、结果百分之百一致"的验证。

7.3 第二层:模型分类准确率——为什么端到端不可复现

判分正确性由两件事共同决定:规则表算得对(第一层已经验证),以及喂给规则表的标签本身判得对。后者只有真实调用模型才能验证,tools/classify_eval.py 用一份人工标注的标准答案集,拿真实的分类接口跑一遍,统计准确率和混淆矩阵。

这里出现了整个项目最容易被误读、也是文档里反复强调"必须写在正文而不是脚注里"的一句话:分类不是确定性的。项目所用的模型上,temperature 参数已经被废弃(无法设置为 0 以获得确定性输出),实测同一份标注集连续跑两次,准确率能相差 0.6 到 1.3 个百分点。这意味着:

- 判分算术(规则表本身)是纯函数,可以离线重跑,结果永远一致;
- 判分标签(某句话命中了哪把钥匙)来自一个会波动的模型,同样的输入不保证同样的输出;
- 因此,严格意义上讲,端到端不可复现。任何验收门槛都要求"看两次跑批",而不是一次跑批定生死。

验收门槛包括:

门槛项	数值
hit_keys 准确率(命中标签集合完全相等)	≥85%
grounded 准确率(全样本)	≥80%
grounded 准确率(仅含钥匙的子集)	≥80%
复读攻略(parrot)子集的 grounded 准确率	≥93%,比通用门槛严得多

为什么复读攻略这一类要单独设一条更严的门槛:如果"扎根门控"这个机制在攻略复读的场景下失灵——也就是说,即使玩家在照抄网络攻略、根本没有真正接住对方说的话,模型却依然把它误判为"扎根"——那么第一层验证出的"攻略传开当天游戏即报废"这道防线在实际运行中就形同虚设。这是唯一一处"总体准确率达标、但某个细分类别已经塌方"仍然会被单独拦下的门槛。

分类器跑批还专门按场景拆分统计准确率——因为"同一套判据、换一个骗局照样成立"是这个项目最核心的主张之一,而如果标注样本三分之二都来自荐股群这一个场景,总体准确率再高也无法证明另外四个场景同样适用,所以必须逐场景单独报告数字,让这句话变成一句可以被证伪的、而不是笼统带过的断言。

这一层的验证成本远高于第一层:每次跑批都要真实调用模型 API,产生费用,因此不能进入 CI 自动跑;而且由于分类结果会波动,单次跑批的结果不能直接作为"通过/不通过"的最终结论,标准做法是跑两次交叉验证。

7.4 第三层:模型演绎质量——量化"AI 味"

判分侧有两把尺子(规则表由蒙特卡洛守着,标签由人工标注集守着),演绎侧原本一把尺子都没有——老陈说话像不像真人、两个不同玩家打出来的老陈是不是"同一个人"、会不会说着说着就暴露出机器味,过去完全靠打几局凭感觉判断。tools/act_eval.py 把这件"凭感觉"的事情变成了可以量化的指标。

它要解决的一个反直觉问题是:靠感觉能发现"像不像人",却发现不了"重不重要"。同一句台词如果在两局之间原样重复出现,单独玩一局的人永远不会察觉,只有把成百上千次调用的输出放在一起统计才能看出来。

它测的核心指标包括:

指标	含义	已知基线数字(改造前 → 人格变体上线后)
首句重复率	同一路线、同一轮次,开场第一句话在不同局之间完全一样的比例	11.6% → 5.6%(某一条路线曾高达 30.4% → 12.5%)
复述反问率	把玩家话里的词原样弹回来当反问,是模型的条件反射而非人的性格	11.2% → 7.5%
四元字组重合率	措辞换了,但句子骨架没变的程度	3.1% → 1.5%
跨路线套话率	同一句台词在完全不同的对话走向里原样出现,说明是模型的固定模板	4.5% → 2.1%
书面语命中率	出现"然而""因此""建议您"这类老陈这个人物完全不会说的书面词汇	0.0% → 0.0%
英文思考过程外漏率	模型的内部思考文本用英文泄漏进最终台词	曾实测约 13 局中有 1 局中招
中文旁白外漏率	模型用"用户""助手"这类元叙述词汇泄漏进玩家能看到的台词	首次测得约 2.0%

为什么"首句重复率"被定为唯一进入硬门槛的核心指标:这个项目最大的卖点是"这是一个活的、会针对你的话做出真实反应的受害者",而不是一段固定剧本。如果两个玩家开局问的是完全一样的问题,却在社交平台上互相截图发现自己拿到的第一句回复一字不差,这个核心卖点会当场破产。目前设定的硬门槛是首句重复率不超过 8%,复述反问率不超过 10%,书面语命中率不超过 2%——这三条门槛一旦踩线,即使其他指标都很漂亮也算不通过。

这一层还有一个专门的检测机制叫**扎根漂移检测**:如果人格变体在某次改版后被调整得会吐出更多具体细节(比如更多的数字、姓名、地点),会导致玩家更容易"接住"这些细节从而触发扎根加成,进而系统性地推高整体胜率——而这条胜率曲线正是第一层蒙特卡洛验证守住的核心资产。因此每次调整人格设定后,都需要用这个机制重新验证扎根率有没有发生漂移,一旦发生漂移,第一层的平衡模拟必须重新跑一遍。这说明第二、三层的改动会反过来影响第一层的结论,四层验证之间不是相互独立的,而是一条会互相牵动的链条。

7.5 第四层:系统并发性能

tools/loadtest.py 测的是上线前唯一一条无法靠单元测试验证的门槛:**50 并发对局同时进行,首句回复延迟的 P95 是否不超过 3 秒**。它打的是真实 HTTP 接口,量的是"从玩家发出这一轮话,到收到第一句可读台词"之间的等待时间——这是玩家能直接感知到的"卡不卡"的那一下,而不是整轮对话全部结束的总耗时。这一层测的不是本机算力,而是模型网关本身能不能撑住 50 路并发请求。

降级率(有多少轮因为模型超时而使用了兜底台词)也会一并报告出来,但不设通过/不通过的门槛——因为降级本身是设计好的正常行为(见第六章"降级不让对局断裂"),真正需要盯的不是降级率是不是零,而是它有没有从"偶尔发生"变成"经常发生"。

7.6 最难的一层:真实干预效果,现在测不了

前面四层——规则表、分类准确率、演绎质量、并发性能——都是这个仓库自己能闭环完成的验证。但产品最终要回答的问题是"它真的能让人在关键时刻多犹豫几秒、少转出去一笔钱吗",而这件事,到目前为止一次都没有被测量过。

原因是结构性的,不是懒得测:

- 核心指标"24 小时内转出放弃率"需要接入真实的交易系统,把干预组用户后续 24 小时内的实际转账行为回传回来做统计——这个仓库本身没有、也不应该有真实交易数据的访问权限;
- 需要一个真实的对照组(现行风险提示弹窗组)才能剥离"这部分用户本来就有一定比例会自己反悔"这个基础放弃率,没有对照组,任何数字都无法解释;
- 场景目前是随机分配给玩家的,而在真实产品里,应当由用户自己触发的异常类型决定分配给他哪一个场景,这套映射逻辑目前也还不存在。

角色对调这个机制并非完全没有外部证据——CHI 2026 发表的 ROLESafe 研究(样本量 144 人,针对中国老年群体,设置了四组对照)发现,"让参与者去扮演劝阻受害者"这一组的诈骗线索识别 F1 分数达到 0.85,是四组里最高的一组,显著优于对照组。但这篇研究测的结局变量是"诈骗线索识别能力",不是"是否真的取消了一笔正在进行的转账",而且该研究里四组之间的自我效能感并无显著差异。因此能够如实陈述的只有一句话:"这个机制有同行评议的实证支持",而绝不能说成"角色对调已经被证明能拦下真实转账"——这两句话之间,隔着这个项目目前全部尚未完成的验证工作。

7.7 四层验证一览表

层次	能否自动化	是否花钱调模型	结果是否确定性	单次耗时	能否进 CI
判分规则表(蒙特卡洛)	能	否	是	秒级	能,每次 push 都跑
分类准确率	能,但需人工标注集	是	否(需两次跑批交叉验证)	几分钟	不能
演绎质量("AI 味")	能	是	否	十几分钟	不能
并发性能压测	能	是(且需服务已启动)	基本是(受网关抖动影响)	数分钟	不能
真实干预效果	不能(需接入真实交易系统)	—	—	至少需 24 小时观测窗	完全无法在此仓库完成

八、两个类比:帮助非技术听众理解复杂度

8.1 判分系统像"给快烧完的柴添火",不是"往火上浇一盆水"

浇水式的劝阻(直接喊"这是诈骗")效果立刻可见,但对方要么被激得更硬(防御姿态升高),要么根本不信——因为这句话别人已经对他说过很多遍。添柴式的劝阻(问用途、拆矛盾、苏格拉底式提问)则必须满足好几个条件才能真正烧起来:得在火苗刚起来的时候(对的情绪档位)添对的柴(对的钥匙),而且得是干柴(扎根,接住他刚说过的原话),同一根柴烧过一次效果就会打折(钝化)。火苗刚起时使劲吹一口气能吹旺(追问窗口),错过那阵风,柴又会重新潮回去。旁边还一直有人在泼油(王老师每 3 轮催一次),火不添柴自己也会慢慢往下掉(信任流失)。

这个类比能一次说清楚为什么这套系统不能做成"选对句子就加分"的简单模型——最终得分不是句子本身决定的,而是"这句话 + 说话的时机 + 有没有接住上文 + 之前有没有说过"这四件事叠乘出来的结果,这也正是判分引擎需要 655 行代码、需要跑两万局蒙特卡洛才敢标定参数的根本原因。

8.2 分类器像"一台每次读数会有细微漂移的秤"

判分规则表是一把标准砝码,永远精确;但喂给这把秤的原始数据(玩家这句话到底命中了哪把钥匙)来自一个每次称重结果都会有细微漂移的仪器——同一批货物过两次秤,读数能差 0.6 到 1.3 个百分点。这不是仪器坏了,而是它的工作原理决定的(模型输出本身带有一定的不确定性,且相关参数已被上游禁用)。

正因如此,任何"这套系统准确率是多少"的问题,负责任的答法都不是甩出一个单次跑批的数字,而是"看两次跑批,取一个区间",这也是为什么第七章反复强调"门槛要看两次跑批"而不是一次。

九、常见追问 Q&A(20 问)

#	问题	标准答法
1	效果验证过吗?降低了多少诈骗损失?	没有,一次都没量过。核心指标"24h 内转出放弃率"需要接入真实交易系统才能测,现在有的是口径定义和对照组设计,不是结果。
2	券商有法定义务做这件事吗?	没有。《反电信网络诈骗法》全文未点名证券公司,义务人是银行和支付机构。推力是投诉与声誉,不是法条。
3	角色对调这个机制有科学依据吗?	有同行评议支持(CHI 2026 ROLESafe,n=144),"去劝受害者"这一组诈骗识别 F1 最高(0.85)。但那篇论文测的是识别能力,不是"取消了一笔转账",不能说成"已被证明能拦下转账"。
4	判分是模型给的还是规则给的?能复现吗?	判分算术 100% 纯函数、可离线重跑、有测试守着。但"命中了哪把钥匙"这个标签由模型分类器给出,同一批标注两次跑批实测差 0.6~1.3 个百分点,所以端到端不算完全可复现,门槛要看两次跑批。
5	换一个模型分数会变吗?	不会。换模型只影响演绎质感和分类准确率,不影响任何分值——这句话可以直接用 <code>classify_eval --model <另一个模型></code> 跑一遍来证伪或证实。
6	老陈只有一种人设,玩两次会不会摸清套路?	全产品共 24 个人格变体(五个场景各 4~8 个),同一局内确定性选定,剧情骨架共享但表达方式、心理支点、方言各不相同,人格差异只影响演绎不改变判分。
7	五个场景都是同一套判分表,那不就是换皮吗?	是同一张表,但不同场景/情绪档位下的最优解至少 2/4 档不同,有专门的测试守着这一点,用来证明"练的是可迁移的能力,不是背某个场景的固定答案"。
8	为什么界面上还写着"投资顾问""李经理",这不是投顾训练器了吗?	刻意保留。产品定位从"投顾话术训练器"转向"C端异动干预"后,坐在这个位置上的人变了(从受训投顾变成异动用户),但判分表一格没动——练的动作是同一套。改词就要动判分闭集,转向明令不许碰它。
9	模型或网关挂了怎么办?	多级降级:演绎失败用分档兜底台词,分类失败按中性判分继续,终局生成失败用预置收尾,对局不会中断,只是台词质量下降。内网不可达时可切公网 provider,但不是自动故障转移,需要人工显式决定。
10	安全性/合规怎么保证不被玩家操纵?	输入侧做"注入吸收",把操纵指令当老陈听不懂的话回应,不转发给模型;输出侧拦截真实股票代码、上市公司名、收益承诺、联系方式、二维码和外链。
11	这是不是就是个聊天机器人套皮的选择题?	不是,玩家是自由文本输入,不是选项。系统靠分类器识别钥匙/失误 + 扎根判定来评估任意自然语言,不是匹配预设选项,因此不容易被"背答案"通关。
12	判分参数是怎么调出来的,靠谱吗?	用蒙特卡洛平衡模拟跑五种不同能力水平的虚拟玩家各两万局,验收门槛(胜率区间、专家上限、复读攻略上限等)是从约 500 组参数扫描出的帕累托前沿反推出来的,不是拍脑袋定的。
13	怎么防止有人在网上传"标准答案"攻略把游戏刷穿?	专门建模了"parrot(复读攻略)"这类虚拟人设,要求它的胜率上限低于 15%,且这一类样本的扎根准确率门槛(93%)比通用门槛(80%)严得多,专门检验"攻略传开当天游戏是否报废"。
14	分数在玩家打的过程中能看到吗?	对局过程中不直接展示命中的方法和分数,只显示情绪档位、剩余轮数和事件,复盘阶段才逐轮揭晓,避免玩家边打边照攻略刷标签。
15	会不会存在某种人格特别容易通关,显得不公平?	人格差异只影响演绎的语气和细节,不改变判分参数;蒙特卡洛跑批时特意让骨架信息(女儿、老伴等)只出现在共享骨架里,不写进单个变体的专属台词,防止某个变体因为多透露信息而系统性更容易触发扎根加成。

#	问题	标准答法
16	这个产品接进真实 App 之后场景怎么分配？	目前场景是随机分配用于演示;真正接入产品后,应当由用户自己触发的异常类型来决定分配给他哪一个场景,这套映射目前还未实现。
17	用户数据(聊天内容)会被留存吗？	默认关闭(<code>TRANSCRIPT_RETENTION=false</code>)。打开后会脱敏留存对局语料,且界面上的告知文案会随开关自动同步显示,不会出现"页面上写的和服务端做的不一致"的情况。
18	系统扛得住多少并发？	已验证 50 并发对局同时进行、首句回复延迟 P95 不超过 3 秒。测的是网关并发额度,不是本机算力,因为等待模型响应期间只是挂起的异步任务。
19	这套东西为什么不能直接拿现有对局去申报,非要包一层"分级触发+处置闭环"的叙事？	因为评审奖励的是"这东西怎么接进真实业务流程",而不仅是"demo 好不好玩"。现有产品最大的空缺是判分只判"信任度",从未判"对话结束后用户有没有真的做安全动作";分级结构和处置闭环是把这个空缺显式补进定位里,而不是重新发明一个概念,且材料中会如实标注这两部分"为设计,非已实现功能"。
20	项目现在最大的短板是什么？	效果指标零测量;单点"接话"练习模式尚未落地;分级触发(Tier 0/1/2)和处置闭环(核验收款方/转人工/安全返回)目前只是设计,代码里不存在;人格方言程度等细节仍需业务专家审核确认。

十、当前完成度与路线图

已实现

- 李经理工作台(企业微信 + 券商 CRM 风格)与微信式对话界面;
- 五个场景(荐股群/冒充公检法/杀猪盘/刷单返利/虚拟币量化)、24 个人格变体、人格一致的开场白;
- 12 轮自由文本对局;
- AI 演绎、AI 闭集分类、确定性的规则判分算术;
- 扎根、钝化、效力矩阵、信任流失、防御姿态、追问窗口、蓄势池、最后一公里、反向施压等全部时机机制;
- 五种终态、终局台词与转账凭证;
- 逐轮复盘、信任度 K 线、全局统计、分享卡;
- 输出安全层、输入注入吸收、多级降级、令牌签名与基础重放防护;
- Windows 本地安装/启动/停止脚本,以及 Linux 部署脚本;
- 四套验证工具:蒙特卡洛平衡模拟、分类器评估、演绎质量评估、并发压测;
- 单元测试覆盖判分引擎、状态机、安全层等核心逻辑。

尚未实现或需要业务确认

- "接话"单点异议应对练习的后端、前端和揭晓页尚未落地,当前入口仍是完整 12 轮对局;
- 15—25 条真实异议题库及每条的高分示范答法,需要业务专家编写和校验;
- 人格方言程度、风控提醒文案、合规存档提示,仍需项目所有者确认;
- 目标服务器上的 50 并发模型网关压测,仍需在真实部署环境完成;
- Redis 为可选配置,未配置时不展示全局统计;
- 分级触发(Tier 0/1/2)与结局后处置闭环(核验收款方/转人工/安全返回交易)目前只是参赛叙事里的设计,尚未开发;
- 核心效果指标(24h 内转出放弃率/撤单率)一次未测量,需接入真实异动流水;
- 五个场景之外,后续可扩展更多客户、骗局和业务阶段。

十一、术语速查表

术语	定义	避免使用的说法
投资顾问	玩家扮演的角色,老陈的开户顾问,受"只能问、不能荐"执业约束	玩家、用户、劝阻者、客服
劝阻对象	AI 扮演的、已被骗局影响的股民,局内叫"老陈"	NPC、角色、骗子、受害者
对局	一局完整的劝阻过程,从开场白到结局,上限 12 轮	会话、session、游戏
接话(尚未落地)	针对单条异议的一次独立应对练习,不累积信任度、不产生结局	单句训练、单轮模式、闯关
信任度	劝阻对象对玩家的信任程度,是决定结局的唯一状态量	分数、好感度、说服值
钥匙	一种被验证有效的劝阻手法,命中即推高信任度	招式、技巧、正向标签
失误	一种会推低信任度的劝阻方式	扣分项、负向标签
扎根	一次命中确实回应对话中出现过的具体内容,而非套用通用模板	相关、grounded、上下文相关性
效力矩阵	每把钥匙在每个情绪档位上的乘数	权重表、系数表
追问窗口	劝阻对象首次晃到更高档位时露出的若干轮机会	连击、combo、机会窗
结局	对局的终态,四档由高到低:劝住、拦下、拖住、转账;被拉黑另计不入档	结果、胜负、outcome
复盘	对局结束后呈现的逐轮回顾,标出转折轮次与错失的钥匙	总结、报告、回放

十二、关键文件索引

文件	内容
README.md	一页对外介绍,含主张边界与本地运行指令
docs/POSITIONING.md	产品身份准绳:核心矛盾、成功标准、不做什么
docs/PROJECT-INTRODUCTION.md	详细业务介绍,本文的主要素材来源之一
docs/EVIDENCE.md	每条外部引用事实的出处,以及它不能支持什么
docs/CONTEST-POSITIONING.md	参赛定位与报名表逐栏定稿
docs/SUBMISSION-CHECKLIST.md	提交当天的操作清单与三条硬边界答法
docs/TECH-DESIGN.md	技术方案施工图
docs/adr/	七条不可轻易反转的架构决策及理由
app/scoring.py	判分引擎,纯函数,可蒙特卡洛离线重跑
app/scenario.py	五个场景:剧本、人格、台词、效力矩阵覆写
app/persona.py	24 个人格变体
tools/balance_sim.py	蒙特卡洛平衡验证(第一层验证)
tools/classify_eval.py	分类器标注集跑批(第二层验证)
tools/act_eval.py	演绎质量与"AI 味"跑批(第三层验证)
tools/loadtest.py	并发压测(第四层验证)
CONTEXT.md	领域术语表,只定义语言,不含实现